

Inferencia Estadística y A/B Testing

¿Es real o es ruido? p-values, experimentos y sus trampas

Enrique Bolaños Abner Xocop

6 de agosto de 2026

Veritas · Libertas · Jvstitia

Agenda

- 1.** La pregunta de la inferencia
- 2.** Estimadores, TCL e intervalos
- 3.** Hipótesis y p-values
- 4.** A/B testing
- 5.** Las trampas
- 6.** Ejercicios
- 7.** Cierre

¿Real o ruido?

Cambiamos el botón de compra de gris a naranja. La conversión pasó de **10.0 %** a **11.5 %**.

Antes de celebrar, la pregunta incómoda: si **no hubiéramos cambiado nada**, ¿qué tan seguido veríamos una diferencia así de grande solo por azar?

Eso es la inferencia estadística: distinguir un efecto del mundo de una casualidad de la muestra. Ayer aprendimos a estimar parámetros; hoy aprendemos a **no creerles demasiado**.

Toda muestra finita produce diferencias sin que nada cambie

Un estimador también es una variable aleatoria

Si repitiéramos el estudio con otra muestra, $\bar{x}$ saldría distinto. Esa variabilidad tiene nombre y fórmula:

$$SE(\bar{x}) = \frac{\sigma}{\sqrt{n}}$$

Dos lecturas importantes del $\sqrt{n}$:

- Para reducir el error a la **mitad** hay que **cuadruplicar** la muestra
- Los rendimientos decrecen rápido: de 100 a 400 datos se gana mucho; de 10,000 a 10,300, nada

Reportar una estimación sin su incertidumbre es reportar la mitad del resultado.

El TCL, visto con simulación

Da igual de qué distribución vengan los datos: el **promedio** de muestras grandes se ve Normal.

```
1 import numpy as np
2 rng = np.random.default_rng(0)
3 # Poblacion nada normal: exponencial, muy sesgada
4 pob = rng.exponential(scale=3, size=1_000_000)
5 medias = np.empty(5000)
6 for i in range(5000):
7     muestra = rng.choice(pob, size=100)
8     medias[i] = muestra.mean()
9 medias.mean(), medias.std()    # ~3, ~0.3 = sigma/10
```

El histograma de medias sale acampanado aunque la población sea una rampa. Por eso la Normal aparece en todos lados.

Intervalos de confianza: qué significan y qué no

Un intervalo del 95 % se construye como $\bar{x} \pm 1,96 \times SE$.

Lo que sí significa

Si repitiéramos el estudio muchas veces, el 95 % de los intervalos contruidos así contendrían el parámetro verdadero.

Lo que NO significa

“Hay 95 % de probabilidad de que el parámetro esté en *este* intervalo”. El parámetro es un número fijo; lo aleatorio es el intervalo, no él.

En la práctica: un intervalo ancho es una confesión de que no tenemos suficientes datos, y eso es información útil.

H0 y H1: el juicio con presunción de inocencia

- H_0 (**nula**): no pasa nada. Las dos versiones del botón convierten igual
- H_1 (**alternativa**): hay una diferencia real

El procedimiento es asimétrico a propósito: asumimos H_0 y solo la abandonamos si los datos son suficientemente incómodos bajo ella.

Nunca “aceptamos” H_0 . Igual que un tribunal no declara inocente sino *no culpable*: puede ser que el efecto exista y no lo hayamos visto con los datos que tenemos.

El p-value, dicho con precisión

$P(\text{datos así de extremos o más} \mid H_0 \text{ cierta})$

Tres cosas que hay que leer despacio en esa frase:

- Es una probabilidad **condicionada a que H_0 sea cierta**
- Incluye lo observado **y todo lo más extremo**
- Es una afirmación sobre **los datos**, no sobre la hipótesis

Si $p = 0,03$: bajo H_0 , resultados así de llamativos ocurren 3 de cada 100 veces.
Eso es todo lo que dice.

El umbral $\alpha = 0,05$ es una convención histórica, no una ley de la naturaleza.

Lo que un p-value NO es

- **No** es $P(H_0 \mid \text{datos})$. Eso requiere un prior (eso fue el martes)
- **No** es la probabilidad de haberse equivocado
- **No** mide el **tamaño** del efecto: con n gigante, una diferencia trivial da p minúsculo
- **No** es replicabilidad: $p = 0,04$ no significa que el estudio se replique el 96 % de las veces
- $p = 0,049$ y $p = 0,051$ **no** son mundos distintos

MÁS ALLÁ DEL CÓDIGO

En 2016 la American Statistical Association publicó un comunicado formal porque el mal uso del p-value estaba distorsionando literaturas científicas enteras.

Errores, potencia y tamaño de muestra

	H_0 es cierta	H_0 es falsa
Rechazo H_0	Error tipo I (α)	Acierto (potencia)
No rechazo	Acierto	Error tipo II (β)

La **potencia** es $1 - \beta$: la probabilidad de detectar el efecto si existe. Sube con más datos y con efectos más grandes.

Por eso el tamaño de muestra se decide **antes** de correr el experimento: se fija el efecto mínimo que valdría la pena detectar y se calcula el n necesario. Decidir el n después es abrir la puerta a todas las trampas de la última sección.

El experimento del botón

- **Control (A):** 4,000 visitantes, botón gris, 400 compras
- **Variante (B):** 4,000 visitantes, botón naranja, 460 compras

Diferencia observada: $0,115 - 0,100 = 0,015$, un aumento relativo del 15 %.

En vez de buscar la fórmula del test de dos proporciones, vamos a **construir la distribución nula a mano**: si el botón no importara, las etiquetas A y B serían intercambiables. Entonces las revolvemos miles de veces y vemos qué diferencias produce el puro azar.

Si puedo simular H_0 , no necesito memorizar la fórmula

Test de permutación desde cero

```
1 rng = np.random.default_rng(7)
2 a = rng.binomial(1, 0.100, 4000)    # control
3 b = rng.binomial(1, 0.115, 4000)    # variante
4 obs = b.mean() - a.mean()
5 todos = np.concatenate([a, b])
6 dif = np.empty(10000)
7 for i in range(10000):
8     rng.shuffle(todos)
9     dif[i] = todos[4000:].mean() - todos[:4000].mean()
10 p_val = np.mean(np.abs(dif) >= abs(obs))
```

`dif` es la distribución nula: lo que produce el azar cuando el tratamiento no hace nada. El p-value es simplemente qué fracción de esas diferencias iguala o supera la nuestra. Sin `scipy`, sin tablas.

A/B testing en la industria

- **Aleatorización real:** se asigna por usuario (no por sesión) y de forma consistente, o el mismo usuario ve las dos versiones y contamina todo
- **Una métrica primaria**, definida por escrito antes de empezar. Las demás son secundarias y no deciden nada
- **Duración fija:** al menos una semana completa, porque el martes y el domingo no se comportan igual
- **Efecto mínimo detectable** declarado antes: si un +0.1 % no cambia ninguna decisión, no diseñen el test para verlo

MÁS ALLÁ DEL CÓDIGO

Un A/B test bien hecho es un experimento aleatorizado controlado, la misma herramienta de los ensayos clínicos.

El peeking problem

Revisar el p-value cada día y parar apenas cruce 0.05 parece eficiente. Es la forma más rápida de inventarse resultados.

```
1 def z(a, b):
2     s = np.sqrt(a.var()/len(a) + b.var()/len(b))
3     return (b.mean() - a.mean()) / s
4 def corre(peek, rng):
5     a = rng.normal(0, 1, 1000)
6     b = rng.normal(0, 1, 1000)    # H0 es CIERTA
7     cortes = range(100, 1001, 100) if peek else [1000]
8     return any(abs(z(a[:n], b[:n])) > 1.96
9                 for n in cortes)
```

Sin peeking: rechaza cerca del 5 % de las veces, como promete α . Con 10 revisiones: alrededor del 20 %. Los datos son ruido puro en ambos casos.

Comparaciones múltiples

Con $\alpha = 0,05$, cada test bajo H_0 tiene 5 % de dar falso positivo. Corran 20 tests independientes y esperen **1 significativo** sin que nada sea real:

$$P(\text{al menos un falso positivo}) = 1 - 0,95^{20} = 0,64$$

Corrección de **Bonferroni**: usar α/m con m tests. Simple, conservadora, honesta.

Tests	α por test	P(algún falso positivo)
1	0.050	0.05
20 sin corregir	0.050	0.64
20 con Bonferroni	0.0025	0.05

Significancia práctica y honestidad

Con $n = 10$ millones, una diferencia de conversión de 0.01 % sale “altamente significativa”. ¿Y qué? No paga ni el sprint que costó.

Siempre reporten **tamaño del efecto** e **intervalo**, no solo el p-value.

p-hacking: las formas de hacer trampa sin mentir

Probar 15 métricas y reportar la que salió; partir por subgrupos hasta encontrar uno significativo; quitar outliers *después* de ver el resultado; parar el test justo cuando conviene; no publicar los experimentos que fallaron.

“Si torturas los datos lo suficiente, confesarán lo que sea”. El antídoto es barato: escriban la hipótesis, la métrica y el n **antes** de mirar nada.

Ejercicio 1: permutación desde cero

Escriban la función completa y úsenla con dos grupos simulados.

```
1 def test_permutacion(a, b, reps=10000, seed=1):  
2     rng = np.random.default_rng(seed)  
3     obs = b.mean() - a.mean()  
4     # ¿Que va aqui?  
5     # 1) concatenar, 2) repetir: revolver y partir,  
6     # 3) guardar la diferencia, 4) fraccion extrema  
7     pass  
8     # g1 = rng.normal(50, 10, 300)  
9     # g2 = rng.normal(52, 10, 300)  
10    # print(test_permutacion(g1, g2))
```

Ejercicio 2: simular el peeking

Midan la tasa real de falsos positivos con y sin peeking.

```
1 rng = np.random.default_rng(11)
2 sin_peek = [corre(False, rng) for _ in range(1000)]
3 con_peek = [corre(True, rng) for _ in range(1000)]
4 # a) tasa de falsos positivos de cada uno
5 tasa_sin = ...
6 tasa_con = ...
7 # ¿Cuántas veces mas alta es la segunda?
8 # Verificacion: tasa_sin ~ 0.05, tasa_con ~ 0.20
```

Ejercicio 3: 100 tests bajo H_0 pura

Ningún efecto existe. ¿Cuántos “descubrimientos” aparecen igual?

```
1 rng = np.random.default_rng(2)
2 significativos = 0
3 for i in range(100):
4     a = rng.normal(0, 1, 500)
5     b = rng.normal(0, 1, 500)    # misma distribucion
6     # ¿Que va aqui? Usen z(a, b) y el umbral 1.96
7     pass
8 # print(significativos)    # esperado: alrededor de 5
9 # Ahora repitan con el umbral de Bonferroni (alpha/100)
```

Resolvemos los tres en el notebook de la sesión; lo que falte queda de práctica para la casa.

Lo esencial de hoy

- Todo estimador tiene incertidumbre: $SE = \sigma / \sqrt{n}$
- El p-value es $P(\text{datos extremos} \mid H_0)$, nada más
- Fijen métrica, n y duración **antes** de correr el test
- **Peeking** y **comparaciones múltiples** inflan los falsos positivos; Bonferroni es la corrección mínima
- Significancia estadística no es significancia práctica
- Si puedo simular H_0 , no necesito memorizar fórmulas

Próxima clase (jueves 13): Naive Bayes

No hay clase el martes 11: Festival de la Libertad